

Exame de época normal: engweb2026-normal

UC: Engenharia Web (3º ano LEI)

Data: 25 de Maio de 2026, 10h30, Ed.1 - sala 1.10

Sinopsis

O objectivo principal deste teste é avaliar os conhecimentos obtidos durante as aulas no desenvolvimento de aplicações Web e outras tarefas afins.

Antes de começares, lê atentamente até ao fim para ficares com uma percepção do todo que se pretende.

Vais ver que tomarás decisões mais acertadas depois de uma leitura completa.

Os resultados finais deverão ser enviados ao docente da seguinte forma:

- Enviar email para: jcr@di.uminho.pt;
- Colocar no subject/assunto: **ENGWEB2026::Especial::Axxxxx**, em que **Axxxxx** corresponde ao número do aluno;
- Enviar ao docente um link do github para um repositório novo criado especificamente para o exame com o seguinte conteúdo (esta preparação poderá valer 1 valor do exame):
 - O repositório no GitHub deverá chamar-se **ENGWEB2026-Normal**;
 - Dentro do repositório deverá haver um ficheiro, **README.md**, contendo uma descrição de como fez a persistência de dados, do setup de bases de dados, respostas textuais pedidas, instruções de como executar as aplicações desenvolvidas, etc. As instruções deverão ser necessárias e suficientes para o docente, a partir do material no repositório, conseguir colocar em execução todos os serviços e testar os requisitos pedidos;
 - Dentro do repositório deverão existir duas pastas: **ex1**, onde colocarão a aplicação desenvolvida para responder ao primeiro exercício e **ex2**, onde colocarão a aplicação desenvolvida para responder ao segundo exercício.

Os exercícios que envolvam criação de rotas serão testados com as rotas no enunciado, qualquer rota que seja diferente da pedida será avaliada com 0.

Recursos

Recursos para a realização da prova:

1. Base de dados sobre Jogos de Tabuleiro (dataset real criado para este exame), este ficheiro tem a seguinte estrutura:

```
[
  {
    "id": "catan",
    "name": "Catan",
```

```
"year": 1995,
"category": "Family",
"minPlayers": 3,
"maxPlayers": 4,
"playingTimeMinutes": 120,
"descriptionEN": "Players take on the roles of settlers, each
attempting to build and develop holdings while trading and acquiring
resources.",
"autores": [
  {
    "id": "klaus-teuber",
    "name": "Klaus Teuber"
  }
],
"editoras": [
  {
    "id": "kosmos",
    "name": "KOSMOS",
    "country": "Germany"
  }
],
"mecanicas": [
  {
    "id": "route-building",
    "name": "Route Building"
  },
  {
    "id": "tile-placement",
    "name": "Tile Placement"
  },
  {
    "id": "dice-rolling",
    "name": "Dice Rolling"
  }
],
"premios": [
  {
    "id": "sdj-1995",
    "name": "Spiel des Jahres",
    "year": 1995
  }
]
},
...
]
```

Exercício 1: Jogos de Tabuleiro (API de dados)

Neste exercício, irás implementar uma API de dados sobre o dataset fornecido. Encontra-se dividido em 3 partes.

1.1 Setup [1 val.]

Realiza as seguintes tarefas seguindo as sugestões para os nomes da base de dados e da(s) coleção(s) fornecidos:

- Analisa o dataset fornecido;
- Introduz as alterações que achares necessárias no dataset. Podes processá-lo e transformá-lo num ficheiro JSON ou vários conforme achares melhor para o que vais ter de fazer;
- Importa-o numa base de dados em MongoDB com os seguintes parâmetros:
 - **database**: deverá chamar-se **jogostabuleiro**;
 - **collection**: deverá haver pelo menos uma de nome **jogos**, poderás criar mais se achares que vai facilitar a gestão da informação.
- Testa se a importação correu bem.

1.2 Queries (warm-up) [0.5 + 0.5 + 1 + 1 + 1 = 4 val.]

Especifica queries em MongoDB para responder às seguintes questões:

- Quantos jogos estão registados na base de dados?
- Quantos jogos pertencem à categoria "Family"?
- Qual a lista de autores (ordenada alfabeticamente e sem repetições)?
- Qual a distribuição de jogos por ano de lançamento (quantos jogos foram lançados em cada ano)?
- Qual a distribuição de jogos por editora (quantos jogos cada editora tem registados)?
- Regista estas queries num ficheiro de texto que deverás colocar na pasta **ex1** chamado **queries.txt**.

1.3 API de dados [0.5 + 0.5 + 1 + 1 + 1 + 1 + 1 + 0.5 + 1 + 0.5 = 8 val.]

Nas rotas pedidas a seguir, é sugerido um determinado formato para a resposta. Se achares útil podes aumentar a informação que é devolvida, acrescentando mais campos ao resultado. As rotas serão testadas tal como constam deste enunciado.

Desenvolve uma API de dados, que responde na porta **17000** e que responda às seguintes rotas/pedidos:

- **GET /jogos**: devolve uma lista com todos os jogos (campos: **id** (ou **_id**), **name**, **year**, **category**, **minPlayers**);
- **GET /jogos/:id**: devolve toda a informação do jogo com o identificador passado na rota (todos os campos);
- **GET /jogos?editora=EEEE**: devolve a lista de jogos que foram publicados pela editora EEEE: **id** (ou **_id**), **name**, **year**;
- **GET /autores**: devolve a lista dos autores, ordenada alfabeticamente por nome e sem repetições (lista de pares: nome do autor, lista de jogos que criou, cada jogo representado por um par: id, nome);
- **GET /categorias**: devolve a lista das categorias, ordenada alfabeticamente e sem repetições (lista de pares: categoria, lista de jogos pertencentes à categoria, cada jogo representado por um par: id, nome);
- **POST /jogos**: acrescenta um registo novo à BD, neste caso, um novo jogo;
- **DELETE /jogos/:id**: elimina da BD o registo correspondente ao jogo com o identificador passado na rota;

- **PUT** `/jogos/:id`: altera o registo do jogo com o identificador passado na rota;
- Acrescenta uma interface **swagger** à tua API de dados;
- Cria a(s) Dockerfile(s) necessária(s) e orquestra tudo num **docker compose**.

Antes de prosseguires, testa as rotas realizadas com o Postman, ou a tua interface swagger ou similar.

Exercício 2: Engenharia Reversa - A Minha Lista de Leituras [7 val.]

Neste exercício apresenta-se uma proposta que terá de ser desenvolvida no sentido inverso ao habitual. Em anexo foi-te fornecido um ficheiro `index.html` que contém uma interface em **Vue.js** para gerir uma Lista de Livros a Ler (Reading List).

Ao analisar o código fonte da interface, detetaste que o frontend faz os seguintes pedidos HTTP usando o Axios:

- **GET** `http://localhost:19020/api/livros`: com possível query string `?search=X`;
- **POST** `http://localhost:19020/api/livros`: enviando um objeto com: titulo, autor, paginas, genero;
- **PUT** `http://localhost:19020/api/livros/:id`: alterando o estado booleano lido;
- **DELETE** `http://localhost:19020/api/livros/:id`: que irá provocar a remoção do registo identificado.

A partir da análise da interface descrita, deverás realizar as seguintes tarefas na pasta `ex2`:

- [1 val.] Derivar o modelo de dados em Mongoose apropriado para suportar esta interface;
- [0.5 val.] Criar um pequeno dataset em JSON exemplificativo (pelo menos 6 registos) para povoar a base de dados inicialmente;
- [2.5 val.] Criar um serviço de API de dados em **Express/Node.js** que implemente rigorosamente os endpoints esperados pela interface;
- [3 val.] Preparar o ambiente para distribuição com Docker:
 - Criar as especificações Dockerfile necessárias;
 - Criar um ficheiro `docker-compose.yml` final que orquestre os serviços.

Requisitos para o docker-compose:

- O servidor MongoDB não deverá estar exposto para o exterior (apenas acessível pela API na rede interna do docker);
 - A API de dados deverá estar exposta ao exterior na porta 19020;
 - Deves servir o ficheiro `index.html` estático usando um servidor Nginx no docker-compose, expondo-o na porta 19021.
-

Bom trabalho e boa sorte, jcr